

Application Protocols for IoT

Dr. Abbas Javed

Assistant Professor

Department of Electrical and Computer Engineering,

COMSATS University Islamabad, Lahore Campus

abbasjaved@cuilahore.edu.pk

The following categories of IoT application protocols and their transport methods are explored in the following sections:

- **Application layer protocol not present:** In this case, the data payload is directly transported on top of the lower layers. No application layer protocol is used.
- **Supervisory control and data acquisition (SCADA):** SCADA is one of the most common industrial protocols in the world, but it was developed long before the days of IP, and it has been adapted for IP networks.
- **Generic web-based protocols:** Generic protocols, such as Ethernet, Wi-Fi, and 4G/LTE, are found on many consumer- and enterprise-class IoT devices that communicate over non-constrained networks.
- **IoT application layer protocols:** IoT application layer protocols are devised to run on constrained nodes with a small compute footprint and are well adapted to the network bandwidth constraints on cellular or satellite links or constrained 6LoWPAN networks. Message Queuing Telemetry Transport (MQTT) and Constrained Application Protocol (CoAP), covered later in this chapter, are two well known examples of IoT application layer protocols.

The Transport Layer

- **Transmission Control Protocol (TCP):** This connection-oriented protocol requires a session to get established between the source and destination before exchanging data. You can view it as an equivalent to a traditional telephone conversation, in which two phones must be connected and the communication link established before the parties can talk.
- **User Datagram Protocol (UDP):** With this connectionless protocol, data can be quickly sent between source and destination—but with no guarantee of delivery. This is analogous to the traditional mail delivery system, in which a letter is mailed to a destination. Confirmation of the reception of this letter does not happen until another letter is sent in response.

- TCP is the main protocol used at the transport layer for human interactions over the Internet.
- TCP is able to transport large volumes of data into smaller sets of packets and ensures reassembly in a correct sequence, flow control, window adjustment, and retransmission of lost packets.
- UDP is most often used in the context of network services or for real-time data traffic, where performance and latency are more important than packet retransmissions.
- Application layer protocols take care of the function of guaranteed error-free packet reception.
- When selecting a transport layer for IoT application layer protocols, the impact on both the lower and upper layers of the stack must be evaluated.

IoT application layer protocols

- IoT application layer protocols are designed to run on constrained nodes with a small compute footprint and are well adapted to the network bandwidth constraints on cellular or satellite links or constrained 6LoWPAN networks. Two well-known examples of IoT application layer protocols are Message Queuing Telemetry Transport (MQTT) and Constrained Application Protocol (CoAP).
- MQTT: MQTT is a lightweight publish/subscribe messaging protocol designed for use on limited bandwidth and unreliable networks. It is commonly used for telemetry and control applications and is supported by a wide range of IoT devices.
- CoAP: CoAP is a simple protocol designed for constrained devices and networks. It is used to transfer data between devices and can operate over both UDP and TCP protocols. It is commonly used for resource-constrained devices, such as sensors and actuators.

- Constrained IoT devices defined as class 0 send or receive only a few bytes of data
- These devices do not implement a fully structured network protocol stack due to processing capability, power constraints, and cost
- Class 0 devices are usually simple smart objects that are severely constrained

Example of Data Transport without Structured Network Stack

- Low-cost temperature and relative humidity (RH) sensors send data over an LPWA LoRaWAN infrastructure
- Small data payload is directly transported on top of the LoRaWAN MAC layer, without the use of TCP/IP
- Lack of standardization makes it difficult for generic implementations of this transport method to be successful from an interoperability perspective

Standardization through IoT Data Broker

- IoT data broker is a piece of middleware that standardizes sensor output into a common format that can be retrieved by authorized applications
- Sensor output can be encoded differently, and the data broker can decode this data into a common, standardized format
- Applications can access this temperature data without having to deal with decoding multiple temperature data formats

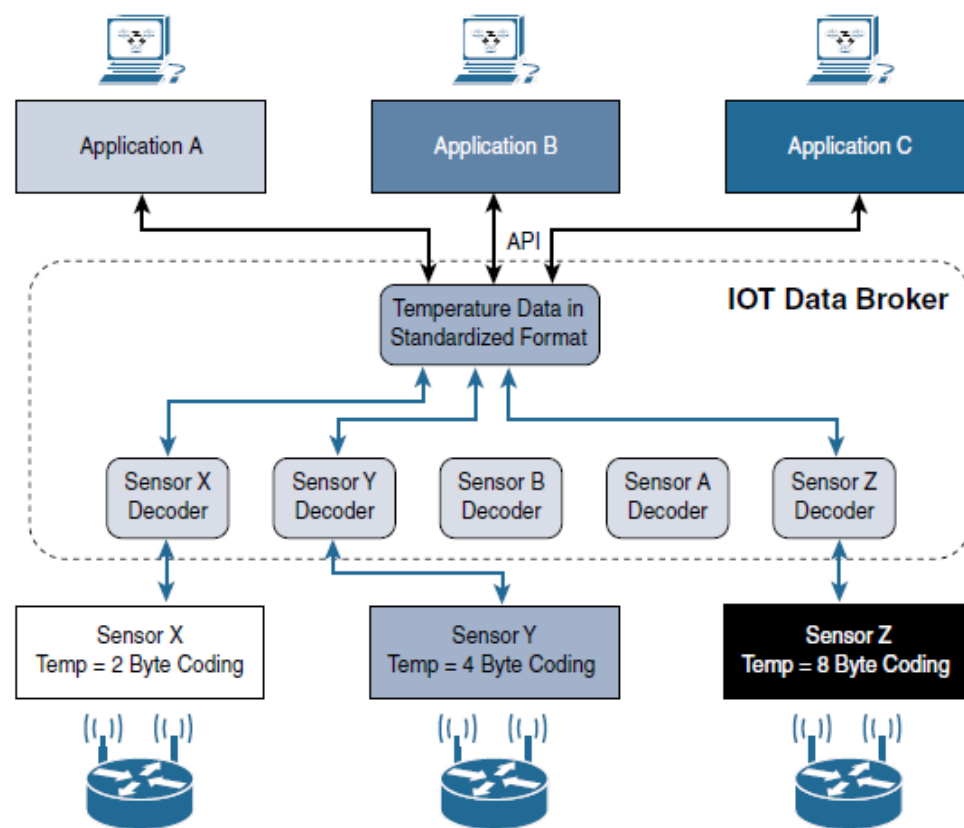


Figure 6-1 *IoT Data Broker*

Commercial Use of IoT Data Broker

- IoT data brokers are utilized from a commercial perspective to distribute and sell IoT data to third parties
- Companies can provide access to their data broker from another company's application for a fee
- This makes an IoT data broker a possible revenue stream, depending on the value of the data it contains

- Directly transporting data payload without a structured network stack optimizes data transmission over low-data-rate networks
- However, lack of a data model implies that each application needs to know how to interpret the data-specific format
- This becomes increasingly complex for larger networks of devices with different data payload formats, calling for structured data models and data broker applications

IoT Application Layer Protocols

- In constrained networks or large-scale deployments of constrained nodes, verbose web-based and data model protocols may be too heavy for IoT applications.
- The IoT industry is working on new lightweight protocols that are better suited to large numbers of constrained nodes and networks.
- Two of the most popular protocols are CoAP and MQTT.

CoAP	MQTT
UDP	TCP
IPv6	
6LoWPAN	
802.15.4 MAC	
802.15.4 PHY	

Figure 6-6 *Example of a High-Level IoT Protocol Stack for CoAP and MQTT*

CoAP Protocol

- Constrained Application Protocol (CoAP) is a protocol designed for constrained networks.
- It is based on RESTful web services and uses the User Datagram Protocol (UDP) for transport.
- CoAP is similar to HTTP, but is designed to work with low-power and lossy networks.
- It is a request-response protocol and supports caching to reduce network traffic.

CoAP Messaging Model

- Designed to facilitate the exchange of messages over UDP between endpoints
- Secure transport protocol Datagram Transport Layer Security (DTLS)
- Studying alternate transport mechanisms including TCP, secure TLS, and WebSocket
- CoAP over Short Message Service (SMS) as defined in Open Mobile Alliance for Lightweight Machine-to-Machine (LWM2M) for IoT device management is also being considered

Security in CoAP

- RFC 7252 provides more details on securing CoAP with DTLS
- CoAP endpoint provisioned with keys and a filtering list
- Four security modes: NoSec, PreSharedKey, RawPublicKey, and Certificate
- NoSec and RawPublicKey implementations are mandatory

CoAP Message Format

- Composed of a short fixed-length Header field (4 bytes), a variable-length but mandatory Token field (0–8 bytes), Options fields if necessary, and the Payload field
- Figure 6-7 details the CoAP message format
- Delivers low overhead while decreasing parsing complexity

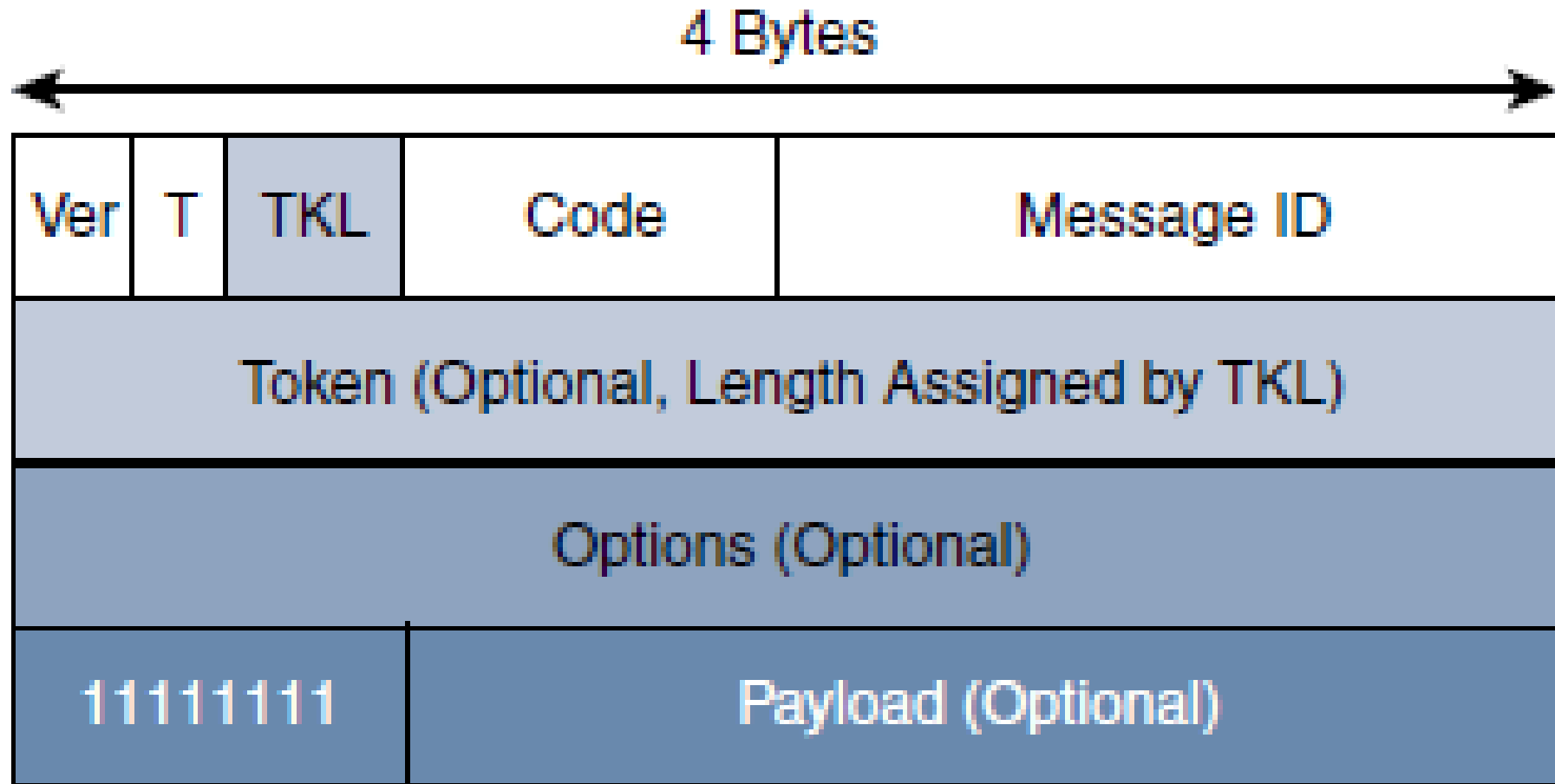


Figure 6-7 *CoAP Message Format*

Table 6-1 *CoAP Message Fields*

CoAP Message Field	Description
Ver (Version)	Identifies the CoAP version.
T (Type)	Defines one of the following four message types: Confirmable (CON), Non-confirmable (NON), Acknowledgement (ACK), or Reset (RST). CON and ACK are highlighted in more detail in Figure 6-9.
TKL (Token Length)	Specifies the size (0–8 Bytes) of the Token field.
Code	Indicates the request method for a request message and a response code for a response message. For example, in Figure 6-9, GET is the request method, and 2.05 is the response code. For a complete list of values for this field, refer to RFC 7252.
Message ID	Detects message duplication and used to match ACK and RST message types to Con and NON message types.
Token	With a length specified by TKL, correlates requests and responses.
Options	Specifies option number, length, and option value. Capabilities provided by the Options field include specifying the target resource of a request and proxy functions.
Payload	Carries the CoAP application data. This field is optional, but when it is present, a single byte of all 1s (0xFF) precedes the payload. The purpose of this byte is to delineate the end of the Options field and the beginning of Payload.

CoAP and IoT Communication

- CoAP can run over IPv4 or IPv6
- Message should fit within a single IP packet and UDP payload to avoid fragmentation
- For IPv6, maximum CoAP message size could be up to 1152 bytes
- For IPv4, implementations should limit themselves to more conservative values and set the IPv4 Don't Fragment (DF) bit

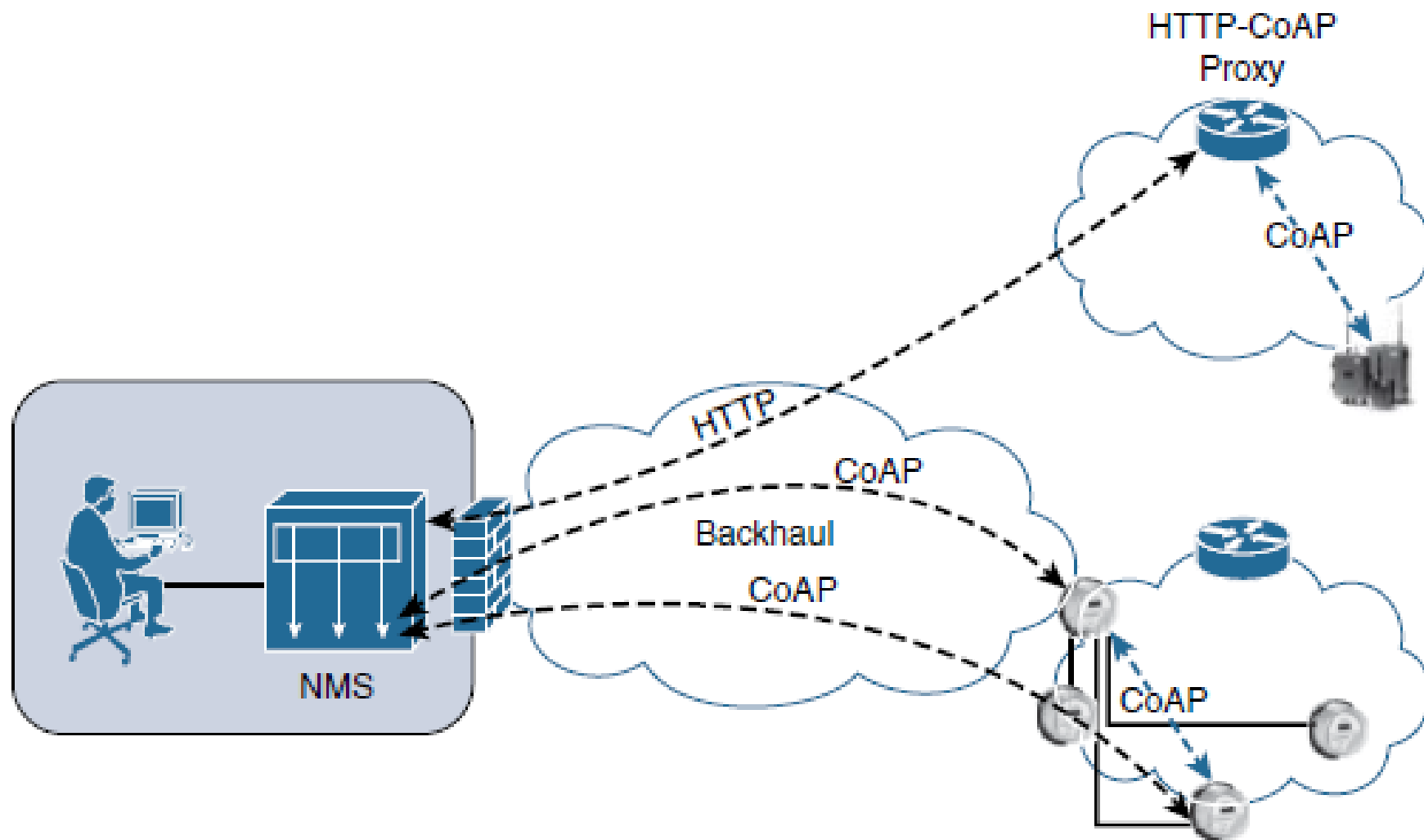


Figure 6-8 *CoAP Communications in IoT Infrastructures*

CoAP and IoT Communication (Contd.)

- CoAP defines a pair of Block options for transferring multiple blocks of information
- CoAP communications across an IoT infrastructure can take various paths
- CoAP is based on the REST architecture, but with a “thing” acting as both the client and the server
- CoAP defines four types of messages: confirmable, non-confirmable, acknowledgement, and reset

CoAP URI Format and Parameters

- Example 6-2 shows the CoAP uniform resource identifier (URI) format, similar to HTTP/HTTPS
- CoAP code, method and response codes, option numbers, and content format have been assigned by IANA as Constrained RESTful Environments (CoRE) parameters
- The CoAP request/response semantics include the methods GET, POST, PUT, and DELETE.

Reliability and Congestion Control in CoAP

- CoAP offers a reliable transmission of messages when a CoAP header is marked as “confirmable”
- CoAP supports basic congestion control with a default time-out, simple stop and wait retransmission with exponential back-off mechanism, and detection of duplicate messages through a message ID

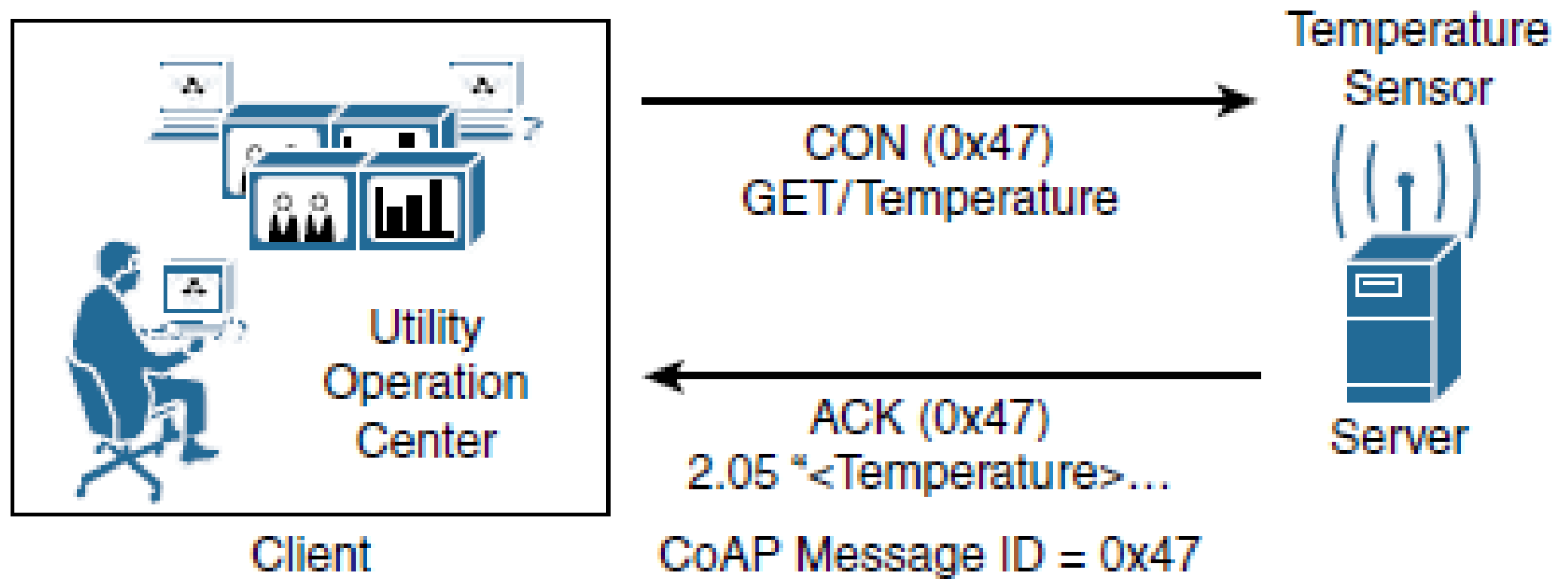


Figure 6-9 *CoAP Reliable Transmission Example*

CoAP Example for Temperature Sensor

- Figure 6-9 shows a utility operations center on the left, acting as the CoAP client, with the CoAP server being a temperature sensor on the right of the figure
- CoAP Message ID ensures reliability and is used to detect duplicate messages
- The client sends a GET message to get the temperature from the sensor, which is acknowledged by the sensor

CoAP Multicast

- CoAP supports data requests sent to a group of devices by leveraging the use of IP Multicast
- Implementing IP Multicast with CoAP requires the use of all-CoAP-node multicast addresses
- CoAP server offering services and resources needs to be discovered by the CoAP clients

CoAP Service Discovery

- Services from a CoAP server can be discovered by learning a URI in a namespace or through the “All CoAP nodes” multicast address
- When utilizing the URI scheme for discovering services, the default port 5683 is used for non-secured CoAP, or coap, while port 5684 is utilized for DTLS-secured CoAP, or coaps

Message Queuing Telemetry Transport (MQTT)- Introduction

- MQTT stands for Message Queuing Telemetry Transport
- Developed by IBM and Arcom in the late 1990s for monitoring and controlling a large number of sensors in harsh environments
- Lightweight, reliable, and cost-effective protocol
- Now standardized by OASIS (Organization for the Advancement of Structured Information Standards)

MQTT Architecture

- Client/server and publish/subscribe framework based on TCP/IP
- MQTT client can act as a publisher to send data to an MQTT server acting as a message broker
- The message broker handles the subscription and unsubscription process and pushes the application data to MQTT clients acting as subscribers
- MQTT decouples the data transmission between publishers and subscribers
- Publishers and subscribers do not have to be online at the same time

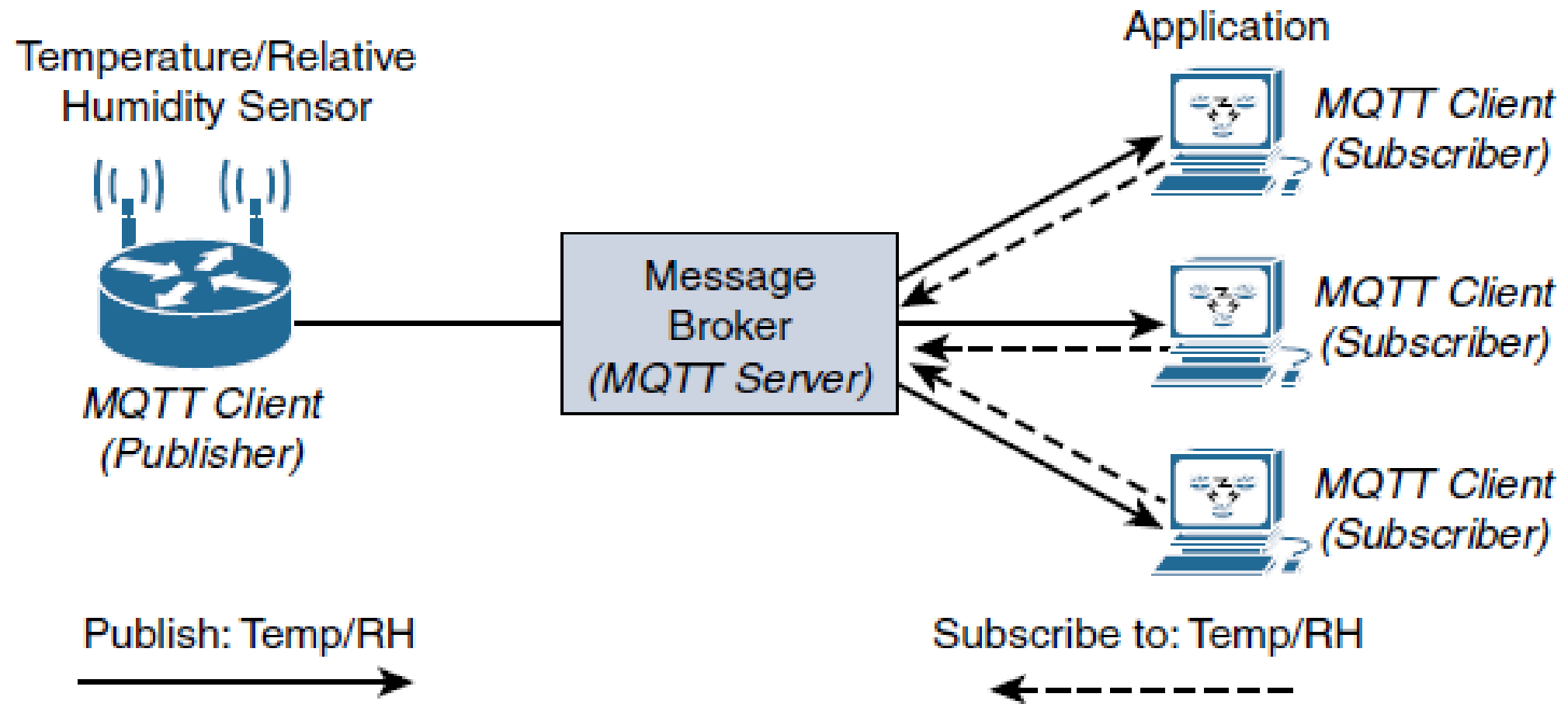


Figure 6-10 *MQTT Publish/Subscribe Framework*

MQTT Wildcards and Subscriptions

- Clients can subscribe to all data or specific data from the information tree of a publisher
- Presence of a message broker ensures that information can be buffered and cached in case of network failures
- MQTT clients can use wildcards to subscribe to all data or specific data
- Subscribers express a desire to receive information from publishers

MQTT Control Packets

- Control packets run over a TCP transport using port 1883
- Optionally, MQTT can be secured using TLS on port 8883, and WebSocket can also be used
- Each control packet consists of a 2-byte fixed header with optional variable header fields and optional payload
- Control packet can contain a payload up to 256 MB

MQTT Message Format

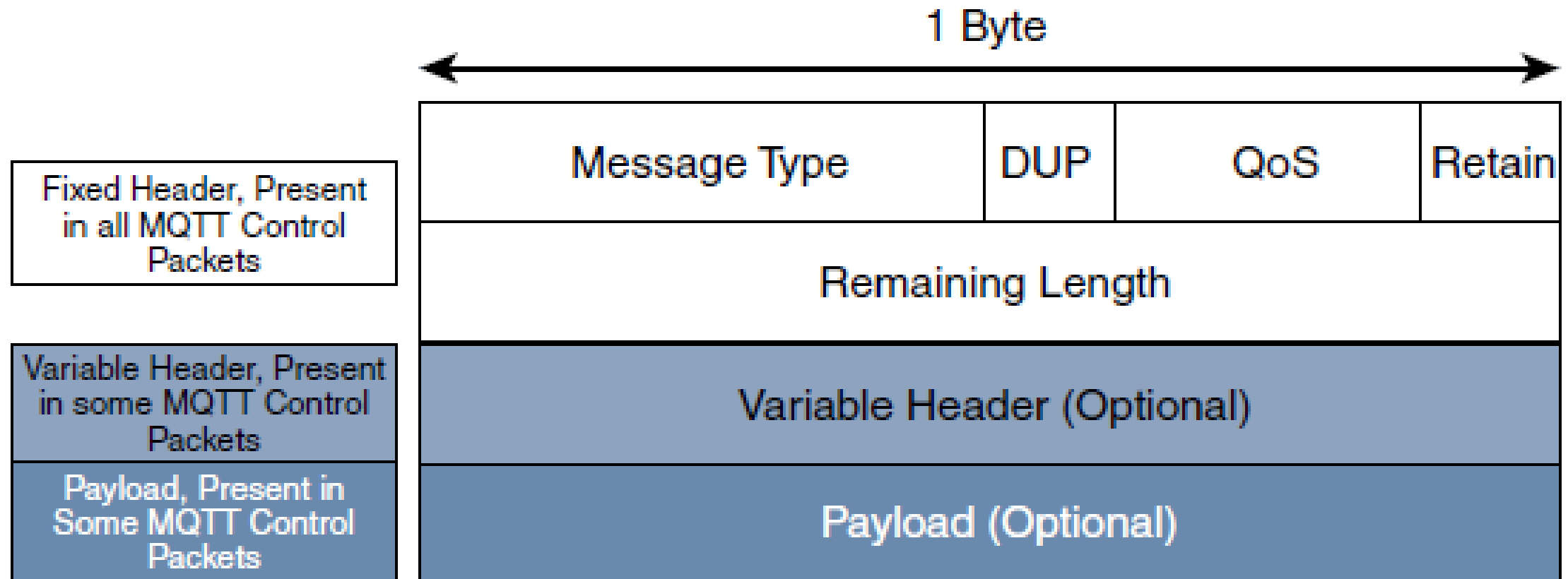


Figure 6-11 *MQTT Message Format*

MQTT Message Format

- Message format overview provided in Figure 6-11
- Contains a smaller header of 2 bytes compared to 4 bytes for CoAP
- The first MQTT field in the header is Message Type, which identifies the kind of MQTT packet within a message
- Fourteen different types of control packets are specified in MQTT version 3.1.1
- MQTT header is DUP (Duplication Flag). This flag, when set, allows the client to notate that the packet has been sent previously, but an acknowledgement was not received

MQTT Message Types

- CONNECT: Client to server request to connect
- CONNACK: Server to client connect acknowledgement
- PUBLISH: Client to server or server to client publish message
- PUBACK: Client to server or server to client publish acknowledgement
- PUBREC: Client to server or server to client publish received
- PUBREL: Client to server or server to client publish release
- PUBCOMP: Client to server or server to client publish complete
- SUBSCRIBE: Client to server subscribe request
- SUBACK: Server to client subscribe acknowledgement
- UNSUBSCRIBE: Client to server unsubscribe request

Table 6-2 *MQTT Message Types*

Message Type	Value	Flow	Description
CONNECT	1	Client to server	Request to connect
CONNACK	2	Server to client	Connect acknowledgement
PUBLISH	3	Client to server Server to client	Publish message
PUBACK	4	Client to server Server to client	Publish acknowledgement
PUBREC	5	Client to server Server to client	Publish received
PUBREL	6	Client to server Server to client	Publish release
PUBCOMP	7	Client to server Server to client	Publish complete
SUBSCRIBE	8	Client to server	Subscribe request
SUBACK	9	Server to client	Subscribe acknowledgement
UNSUBSCRIBE	10	Client to server	Unsubscribe request

Message Type	Value	Flow	Description
UNSUBACK	11	Server to client	Unsubscribe acknowledgement
PINGREQ	12	Client to server	Ping request
PINGRESP	13	Server to client	Ping response
DISCONNECT	14	Client to server	Client disconnecting

- Three Type of QoS
- QoS 0
 - Best-effort and unacknowledged data service
 - No response or retry
 - Message arrives once or not at all
- QoS 1
 - Message delivery occurs at least once
 - Packet identifier in variable header
 - PUBACK packets
- QoS 2
 - Message delivery occurs exactly once
 - PUBREC, PUBREL, and PUBCOMP packets
 - Packet identifier in variable header

- Retain flag
 - Purpose of Retain flag
 - Found only in PUBLISH message
 - Notification to server to hold onto message data
- MQTT sessions
 - Four phases of MQTT sessions i.e., session establishment, authentication, data exchange, and session termination.
 - Unique client ID for each client connecting to server
 - Delivery of application message to multiple clients
- Subscriptions and unsubscriptions
 - SUBSCRIBE/SUBACK control packets for subscriptions
 - UNSUBSCRIBE/UNSUBACK control packets for unsubscriptions
 - DISCONNECT control packet for graceful termination of connection

Topic strings and topic names

- How message broker uses topic string to filter messages
- Hierarchical structure of topic names
- Example of a topic name: `adt/lora/adeunis/0018B20000000023A`

Wildcard characters

- Subscription to multiple topics using wildcard characters
- Pound sign (#) matches any number of levels
- Plus sign (+) matches only one topic level
- Excluding topic names beginning with dollar sign (\$)

Conclusion

- MQTT is a lightweight and reliable protocol for monitoring and controlling sensors in harsh environments
- MQTT uses a client/server and publish/subscribe framework based on TCP/IP
- MQTT clients can subscribe to all data or specific data using wildcards
- MQTT control packets consist of a 2-byte fixed header with optional variable header fields and optional payload
- Fourteen different types of control packets are specified in MQTT version 3.1.1.

Table 6-3 *Comparison Between CoAP and MQTT*

Factor	CoAP	MQTT
Main transport protocol	UDP	TCP
Typical messaging	Request/response	Publish/subscribe
Effectiveness in LLNs	Excellent	Low/fair (Implementations pairing UDP with MQTT are better for LLNs.)
Security	DTLS	SSL/TLS
Communication model	One-to-one	many-to-many
Strengths	Lightweight and fast, with low overhead, and suitable for constrained networks; uses a RESTful model that is easy to code to; easy to parse and process for constrained devices; support for multicasting; asynchronous and synchronous messages	TCP and multiple QoS options provide robust communications; simple management and scalability using a broker architecture
Weaknesses	Not as reliable as TCP-based MQTT, so the application must ensure reliability.	Higher overhead for constrained devices and networks; TCP connections can drain low-power devices; no multicasting support